

Enunț

Întrucât aplicațiile anterioare au avut un succes extraordinar, organizația Q&V dorește să își extindă domeniul de activitate deschizând un restaurant. Pentru a eficientiza operațiunile, aceștia doresc să implementeze o aplicație.

Aplicația va permite adăugarea comenzilor clienților. Fiecare comandă are un identificator unic și un număr dinamic de produse. Fiecare produs are un nume și un gramaj. Fiecare produs poate fi fie o băutură (memorându-se dacă este la sticlă sau nu), desert (menționându-se formatul servirii, felie, cupă sau porție) sau burger (listând o serie de ingrediente, de exemplu roșii, varză, etc.). Produsele sunt predefinite și se găsesc în cadrul unui meniu. Pentru fiecare dintre ele se poate calcula valoarea de energie necesară pentru prepararea cu formula:

- Băutură -> dacă este la PET, 25, dacă nu $0.25 * \text{gramaj}$
- Desert -> dacă este felie 25, porție $0.5 * \text{gramaj}$ sau cupă $2 * \text{gramaj}$
- Burger -> $\text{gramaj} * 0.25 * \text{numărul de ingrediente}$

Pentru a demonstra funcționalitatea aplicației va exista un simulator. Acesta este construit pe baza unui ciclu ce include ca toți angajații să participe în sarcinile magazinului. Acestea sunt: preluarea, prepararea și livrarea comenzilor. Întrucât Q&V dorește să aplice tehnici de management modern, inspirate din IT, aceștia vor să aibă doar echipe cross funcționale, astfel toți angajații pot să facă toate acțiunile, însă aceștia au eficiență diferită.

Un angajat are un număr de puncte de energie, inițial 100. Aceștia consumă energie pentru a face o anumită acțiune. După un ciclu, se întorc înapoi la 100 energie.

La începutul unui ciclu se introduc comenzile. Pentru a lua o comandă, un angajat consumă 100 de puncte de energie. Un angajat poate să preia maxim o comandă.

După etapa de preluare a comenzilor, sunt preparate comenzile. Angajații își consumă energia pentru a prepara comenzi. (De exemplu, un angajat cu 100 energie va scădea 100 din energia necesară preparării comenzii).

Dacă o comandă este terminată, atunci ea poate fi înapoiată către client la un cost de 100 energie.

Există mai multe tipuri de angajați, fiecare având anumite perk-uri:

- Casieri => cost de energie doar de 25 pentru a prelua o comandă
- Livratori => cost de energie doar de 25 pentru a livra o comandă
- Bucătari => dublează cantitatea de energie pe care pot să o producă asupra unui produs, iar aceștia primesc 100 energie la finalul ciclului în loc să întoarcă la 100 (aka pot depăși acel 100 energie inițial).

Cerințe:

- Afișarea numărului de angajați pentru fiecare tip în parte
- Implementarea simulării și afișarea unui ciclu (sub orice fel).
- Crearea unei opțiuni ca între cicluri să poți selecta comenzi care sunt prioritare, și ar trebui să fie terminate primele
- Optimizarea ciclului (se va prioritiza ca fiecare angajat să își facă rolul, vezi exemplu pentru varianta neoptimizată)
- Posibilitatea de a selecta strategia de alegere a comenzilor care sunt preparate (de exemplu FIFO, comanda cea mai "grea" prima, comanda cea mai ușoară prima)

Atenție! Precizări pe verso

Exemplu orientativ:

```
angajati = [Livrator(100), Bucătar(125), Bucătar(100), Casier(100)]
           // un bucătar are 125 întrucât a rămas cu 25 energie la ciclul
           precedent
comenzi = []
== START ==
Adăugare comanda? (1/0) 1
comenzi = [Comanda(650 energie)] // Livrator citește comanda devine Livrator(0)
Adăugare comanda? (1/0) 0 // la primul refuz, se vor evita adăugări ulterioare
comenzi = [Comanda(400)] // Bucatar consuma energie devine Bucatar(0)
comenzi = [Comanda(200)] // Bucatar consuma energie devine Bucatar(0)
comenzi = [Comanda(100)] // Casier consuma energie devine Casier(0)
== END ==
angajati = [Livrator(100), Bucătar(100), Bucătar(100), Casier(100)]
== START ==
Adăugare comanda? (1/0) 0
comenzi = [Comanda(0)] // Livrator consuma energie devine Livrator(0)
comenzi = [] // Bucatar livreaza comanda, devine Bucatar(0)
== END ==
angajati = [Livrator(100), Bucătar(100), Bucătar(200), Casier(100)]
```

Precizări:

0. Sursa predată trebuie să compileze. Erorile mici se pot trece cu vederea. Înainte de predarea surselor, studenții vor pune în comentarii eventualele părți din program care au erori de compilare sau nu funcționează corespunzător. Codul comentat se poate puncta parțial dacă este corect.
1. Sursa predată trebuie să respecte condițiile expuse în **Regulamentul de colocviu** care a fost trimis către studenți (via Teams) în data de 01/06/2025.
2. Se acceptă și soluții parțiale, care nu respectă toate cerințele din enunț, dar sunt funcționale. Acestea vor fi punctate corespunzător.
3. În implementarea programului se vor utiliza cât mai multe dintre noțiunile de programare orientată pe obiecte, care au fost studiate pe parcursul semestrului și care se potrivesc cerințelor din enunț.
4. Orice tentativă de fraudă se va pedepsi conform regulamentelor Universității.
Se interzice utilizarea sau rularea în fundal a aplicațiilor care pot fi folosite pentru a comunica. În cazul observării acestora, nota la examen va fi 1.
5. Dacă rezolvarea nu este făcută cu clase sau dacă datele din clase sunt publice, nota va fi 1.
6. Funcționalitățile aplicației vor fi implementate la alegere sub formă de meniu interactiv sau ca apeluri în funcția main pentru fiecare cerință.

Barem POO 02.06.2025

1p - oficiu (sursa compilează)

2p - ierarhie (identificarea claselor + atributelor)

De exemplu:

Produs (nume, gramaj)

| Bautura (pet)

| Desert (tip)

| Burger (ingrediente)

Comanda (id, vector<pair<Produs, int>>)

| se accepta și doar vector<Produs>

Angajat (energie)

| Casier

| Bucatar

| Livrator

Se acceptă și alte soluții cât timp îndeplinesc principiile de bază ale POO.

0.25p - const (utilizarea const pentru attribute, funcții sau argumente la funcții)

Ideal, minim într-un loc unde nu este cod generat.

0.25p - static (folosirea de funcții și attribute statice)

0.25p - destructor virtual

Nu se punctează dacă nu se folosește în toate locurile în care este necesar.

0.25p - moștenire (reutilizare de cod)

De exemplu, apelarea unei funcții a clasei de bază în momentul rescrierii ei în clasa derivată.

0.25p - abstract (existența a unei clasei abstracte)

0.25p - existența unei funcții ce calculează energia necesară pentru un produs.

Se punctează complet dacă este funcție virtuală, rescrisă corespunzător în derivate

0.25p - existența unei funcții care calculează energia unei necesare pentru o comandă.

Se punctează doar dacă există o clasă Comanda implementată corespunzător

0.25p - folosirea STL containers

De exemplu, vector, list, map, etc.

0.25p - folosirea din STL a altor elemente diferite de containers

De exemplu, sort, transform, swap.

0.25p - aruncarea unei excepții

Se punctează doar dacă excepția se propagă (nu poate fi tratată în același loc de locul unde a fost aruncată)

0.25p - prinderea unei excepții

Tratarea cu sens a unei excepții aruncate

0.25p - validarea tipului unui desert

În constructor se validează că tipul unui desert este doar cel prestabilit.

Se punctează și dacă este folosit un enum.

0.25p - RTTI

Utilizarea dynamic_cast sau type_info

0.25p - Existența unei clase Singleton

0.25p - Existența unei clase Factory

1p - identificarea comportamentului de bază al clasei angajat

Minim, funcții care întorc costul de bază pentru preparare, livrare, comandare, recuperare.

Alte funcții precum cele care verifică dacă o acțiune poate fi executată.

0.75p - identificarea comportamentului claselor derivate ale clasei angajat

Rescrierea funcțiilor în clasele derivate ca să fie evidențiat comportamentul specific al derivatelor.

De exemplu, funcția de recuperare trebuie rescrisă în clasa Bucătar.

0.5p - asamblarea comportamentului clasei Angajat

Funcții care îmbină comportamentul de bază + verificările necesare ca să creeze clasa Angajat

0.25p - ID comandă incrementat automat în constructor

0.5p - existența unui meniu cu produse predefinite

0.5p - afișarea numărului de angajați de fiecare tip.

Se punctează complet doar dacă este folosit `dynamic_cast` sau `type_info`

1p - implementarea ciclului dat ca exemplu

1p - prioritizarea unei comenzi

Se presupune că implementarea naivă a ciclului va lucra prima comandă disponibilă dintr-un vector.

Prioritizarea înseamnă existența unei opțiuni în meniu în care pot introduce un ID al unei comenzi care va prioritiza ca o anumită comandă să fie preparată / livrată prima.

0.5p - optimizarea ciclului

Se presupune că ciclul iterează naiv prin angajați.

Varianta optimizată va accesa angajații astfel încât să aloce acțiunile specifice către angajații cu o anumită funcție

1p - strategia de alegere a comenzilor care sunt preparate

Se presupune că implementarea naivă a ciclului va lucra prima comandă disponibilă dintr-un vector.

Varianta pentru acest subpunct ar trebui să nu mai aleagă prima comandă, ci una conform criteriului ales.

Se punctează maxim dacă se implementează conform Strategy Pattern.

0.5p - bonus (pentru folosirea conceptelor POO care nu sunt în barem, precum template)

-0.5p - depunctări (de exemplu memory leaks)

Total: 14p

ATENȚIE! Nu se punctează lucrurile care nu sunt folosite cu sens conform enunțului.
(de exemplu, aruncarea unei excepții în momentul citirii opțiunilor din meniu)